

Improving Scalability of a Scientific Cloud Platform Using Object Storage

Capstone Project Final Report

Artur Rafael da Rocha Moreira
Casemiro Melo Jorge de Medeiros
Guilherme José Alves de Oliveira e Sá
José Sebastião Vizcaíno Pereira



Bachelor in Informatics and Computing Engineering

U.Porto Supervisor: Ademar Aguiar
Project Proponent: Artur Rocha

05 July 2026

Contents

1	Introduction	3
1.1	Context	3
1.2	Objectives and Expected Results	3
1.3	Report Structure	4
2	Methodology and Activities	4
2.1	Utilized Methodology	4
2.2	People Involved	5
2.3	Developed Activities	6
2.3.1	Deliverables	7
3	Development	8
3.1	Requirements	8
3.1.1	Functional Requirements	9
3.1.2	Non-Functional Requirements	9
3.1.3	Constraints	10
3.2	Architecture and Technologies	11
3.2.1	Technology Stack	11
3.2.2	Implementation	11
3.3	Developed Solution	15
3.3.1	Dataset Upload and Visual Feedback	15
3.3.2	Resuming Interrupted Uploads	16
3.4	Validation	17
4	Conclusions	18
4.1	Achieved Results	19
4.2	Learned Lessons	19
4.3	Known Limitations	20
4.4	Future Work	21
4.5	Related Sustainable Development Goals	21

List of Figures

1	Gantt chart illustrating the timeline of the developed activities and key milestones from late February to early June.	8
2	UML Sequence Diagram illustrating the coordinated upload workflow between the Angular frontend, FastAPI backend, and MinIO storage layer.	13
3	High-level system architecture illustrating the components of the platform and the direct client-to-storage communication path. . .	15
4	The dataset upload interface displaying a live progress bar during a massive file transfer.	16
5	The recovery prompt presented to the user upon returning to an interrupted upload session.	16

List of Tables

1	Functional Requirements for development	9
2	Non-Functional Requirements for development	10
3	Constraints for development	10
4	Test cases with descriptions and results	18

1 Introduction

This report aims to present the work developed throughout the semester for our capstone project. It provides the general context in which the project was developed, details the main problem we aimed to solve, and describes the development of our solution, including its design, implementation and evaluation.

1.1 Context

Our project took place at INESC TEC (Institute for Systems and Computer Engineering, Technology and Science), a research & development (R&D) institute at the Faculty of Engineering of the University of Porto (FEUP). INESC TEC currently incorporates thirteen different R&D centres [1], but our project was within HumanISE (Human-Centered Computing and Information Science), which fosters the development of software systems, methods, and tools designed to empower individuals and their communities [2]. One of the many initiatives developed by HumanISE is VAXPRED, a platform part of the Inno4Vac consortium utilized by scientists designed to predict vaccine immunogenicity, which relies heavily on large-scale datasets generated by next-generation sequencing (RNA-seq) [3, 4]. This specific platform is the core focus of our work.

Having established the context, we can now address the problem assigned to our project: the platform’s data management and storage capabilities. Each RNA-seq dataset used in the platform can be extremely large, and it is relatively common for each of these large-scale datasets to exceed the 50 GB mark. Because of this, scaling the platform’s upload and download capabilities proves to be a challenge. The current approach is to store all files directly in the server’s filesystem. This design approach quickly becomes a bottleneck and is very difficult to scale, since the storage capacity is directly limited to the server/hosting machine. Additionally, it brings another layer of problems, since large transfers to the server can saturate the disk’s I/O response and compromise the system’s responsiveness, as well as other platforms used by the institution that need access to said server.

To address these limitations, a cleaner and more robust approach would involve decoupling data storage from the application layer by adopting an object storage solution, of which we opted for MinIO [5], due to its high-performance and compatibility with the S3 (Amazon Simple Storage Service) API [6]. This way, by moving these datasets from the local filesystem to MinIO, the platform can scale the storage independently from the application server, effectively removing the performance bottleneck and the scalability constraint.

1.2 Objectives and Expected Results

The main objective of this project is to analyze and prototype a proof of concept for integrating an object storage system, namely MinIO as previously mentioned, into the existing web application to optimize the handling of the aforementioned large files. The proposed solution aims to validate the feasibility of

replacing the current filesystem-based storage approach with an object storage architecture while preserving the platform’s existing functionalities. In doing so, the project seeks to evaluate whether this architectural change can provide a more scalable, efficient, and maintainable solution for storing and managing the large RNA-seq datasets handled by the platform.

Consequently, the expected outcome is a functional prototype integrating MinIO into the existing platform. This includes the refactoring and extension of the platform’s API through the addition of new endpoints required to support object storage operations, as well as improvements to the database schema to provide a more robust and comprehensive representation of file metadata. Through these developments, the prototype will demonstrate the feasibility of adopting an object storage architecture while maintaining compatibility with the platform’s existing workflows.

1.3 Report Structure

The remainder of this report is organized into three main parts. The first presents an overview of the software development methodologies and engineering practices adopted throughout the project. It also identifies the key stakeholders and collaborators involved during the semester, outlining their roles and contributions to the project’s development. The second describes the details of the development itself, including the architecture, technologies used, an overview of the solution developed, and validation. Finally, the last part summarizes the project’s outcomes, reflects on the knowledge and experience gained throughout its development, discusses the limitations of the proposed implementation, and presents recommendations for future improvements and possible extensions.

2 Methodology and Activities

With the organizational context and project objectives established, this section details the practical approach taken to execute the work. Expressly, it outlines the development methodology adopted, introduces the key stakeholders and individuals involved, and describes the concrete activities undertaken to prototype the integration of the MinIO object storage into the VAXPRED platform.

2.1 Utilized Methodology

Throughout the development of the project, the team held weekly meetings every Friday with the proponent of our project, Artur Rocha, and a member of IN-ESC TEC, Alexandre Costa. These meetings served as the primary mechanism for monitoring the project’s progress, discussing technical challenges, clarifying functional and non-functional requirements, and defining the objectives for the following development cycle.

Although the project did not strictly follow a formal Agile framework, the development process adopted an iterative approach that closely resembled sprint-

based development. Each weekly cycle consisted of reviewing the work completed during the ongoing week, gathering feedback from the supervisors, prioritizing the next set of tasks, and planning the activities to be carried out before the subsequent meeting.

Regarding the development resources, the project was based on the existing codebase hosted on INESC TEC’s GitLab [7] instance, which was used for version control and collaborative development. Notably, rather than employing a monolithic structure [8], the application follows a decoupled approach [9] where the frontend and backend are maintained as two distinct repositories. As the work involved extending and adapting both the frontend and backend components of the application, this separation facilitated independent versioning and parallel development while keeping a cleaner version history.

2.2 People Involved

The project team consisted of four members: Artur Moreira, Casemiro Medeiros, José Vizcaíno, and Guilherme Sá. Although all team members contributed across most areas of the project, responsibilities were naturally divided according to each member’s primary focus. Artur Moreira and Guilherme Sá were mainly responsible for the backend development and database-related tasks, including the integration of the object storage solution and the necessary modifications to the application’s API and data model. Casemiro Medeiros and José Vizcaíno focused primarily on the frontend, implementing the user interface changes required to support the new file management workflow. They also contributed to improving the application’s portability and configuring the necessary runtime requirements. Larger features were often developed collaboratively, with multiple team members working together on different components before integrating the final solution.

The project was carried out under the academic supervision of Professor Ademar Aguiar, while the industrial supervision was provided by Artur Rocha from INESC TEC, with additional technical support from Alexandre Costa. Weekly meetings were held throughout the semester to review the team’s progress, discuss implementation challenges, validate architectural and technical decisions, and define the objectives for the following development cycle. This continuous feedback process ensured that the proposed solution remained aligned with the project’s objectives and the needs of the host institution.

The primary stakeholders of the project were the researchers involved in the VAXPRED platform at INESC TEC. Although the development team did not interact directly with these stakeholders, communication was facilitated through Artur Rocha, who acted as the liaison between the researchers and the student team. Functional requirements, feedback, and new requests were gathered from the stakeholders and communicated to the team during the regular project meetings.

The main responsibility assigned to the team was to investigate and prototype the integration of an object storage solution based on MinIO into the existing VAXPRED platform. This involved evaluating the feasibility of replac-

ing the filesystem-based storage approach, extending and adapting the backend API, modifying the database schema to support enhanced file metadata, and updating the frontend to accommodate the new storage workflow while preserving the platform’s existing functionality.

2.3 Developed Activities

The project was executed over the course of the semester, officially beginning on February 20th and concluding in early June. The work was divided into sequential phases that reflect an iterative development process. The activities were guided by the weekly Friday meetings with the project supervisors, which served as the main checkpoints for validating progress and defining upcoming tasks.

The main activities carried out during the project can be grouped into the following phases:

Phase 1: Project Setup and MinIO Exploration (Late February – Early March)

The initial phase focused on understanding the core problem of handling massive datasets and exploring object storage solutions. The team dedicated this period to studying MinIO, understanding the S3 API standard, and defining how an object-storage architecture could be mapped to the existing VAXPRED platform requirements.

Phase 2: Backend Architecture and Storage Integration (Mid-March – Mid-April)

Following the initial exploration, the team began working directly with the backend repository. Modern dependency management was introduced using `uv` to ensure reproducible environments. During this phase, the database schema was updated using Alembic [10] to support extended file metadata (e.g., bucket names, upload status). The core FastAPI endpoints for creating, retrieving, and deleting datasets were successfully refactored to coordinate with a locally deployed MinIO instance.

Phase 3: Frontend Adaptation and Workflow Merge (Late April – Mid-May)

With the backend storage logic established, the team shifted focus to the frontend repository. The Angular application was adapted to support the new dataset architecture, shifting the file transfer workflows to utilize direct S3 transfers via presigned URLs. This phase culminated in a stable merge of the backend and frontend components, ensuring that existing platform features, such as sample associations, remained functional.

Phase 4: Advanced File Management and Fault Tolerance (Late May – Early June)

To address the specific challenges of transferring datasets exceeding 50 GB, the team implemented a multipart upload mechanism. Subsequently,

significant effort was dedicated to developing fault-tolerance features, including a resumable upload system to recover from interrupted sessions, and conflict-handling modals to prevent accidental file overwrites.

Phase 5: Desktop Porting and Stabilization (Early June)

Although identified as a secondary objective, the team successfully wrapped the Angular application within an Electron container to provide a native desktop experience. This phase involved resolving cross-origin authentication issues, specifically adapting the Keycloak [11] redirect flow to function properly within the standalone desktop window.

Phase 6: Validation, Testing, and Documentation (Early June)

The final phase was dedicated to end-to-end manual testing, bug fixing, and refining the user interface. Simultaneously, the team compiled the technical documentation, drafted this Final Report, and prepared the final presentation slides for the capstone project evaluation.

2.3.1 Deliverables

Throughout the execution of the project, the primary deliverable produced was the fully functional proof of concept, which was successfully integrated into INESC TEC's Gitlab. This overarching deliverable consists of the following specific technical artifacts:

- **Backend and Storage Infrastructure:** The extended FastAPI backend repository, featuring the new centralized storage service, the updated Alembic database migrations, and the Docker Compose orchestration required for the local MinIO deployment.
- **Frontend Application:** The refactored Angular repository, adapted to support direct S3 object transfers, complete with the newly developed user interface components for progress visualization, conflict resolution, and resumable multipart uploads.
- **Standalone Desktop Client:** The experimental Electron port of the platform, packaged to provide a native desktop experience with custom authentication routing.

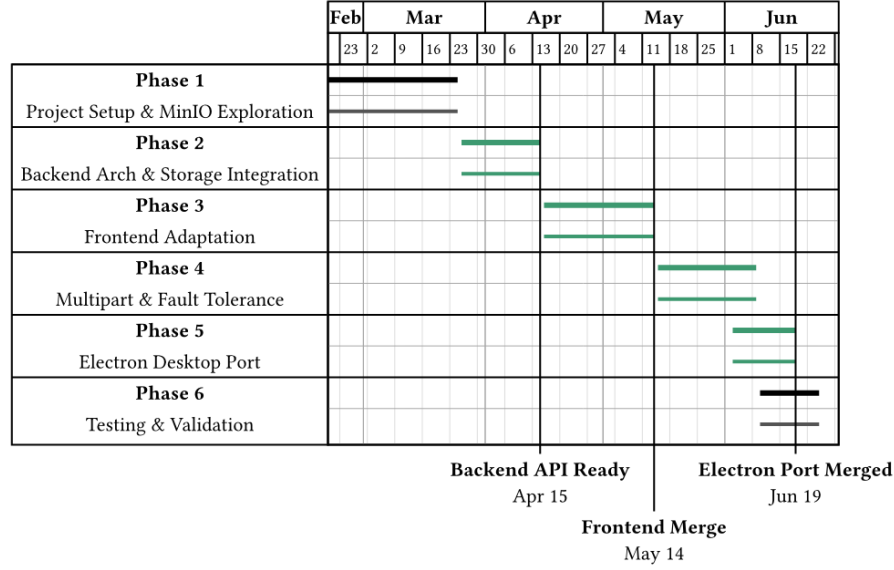


Figure 1: Gantt chart illustrating the timeline of the developed activities and key milestones from late February to early June.

3 Development

With the methodology and foundational activities established, this section details the core engineering phase of the project, translating the proposed concepts and their associated constraints into a functional prototype. The chapter begins by defining not only the specific requirements necessary for integrating the MinIO object storage but also the additional project objectives addressed during development, ensuring the platform aligns with its existing guidelines. Subsequently, it presents the system architecture and the utilized technology stack. Following this, an overview of the developed solution is provided. This subsection details technical challenges encountered during implementation and the strategies adopted to overcome them. After that, there is a description of the final state of the prototype. Finally, the chapter concludes by detailing the validation procedures conducted to ensure the new workflow meets the project's stakeholders needs and INESC TEC's standards and expectations.

3.1 Requirements

The requirements for this project were not formally specified from the get-go. Instead, they were defined throughout the initial stages of development based on meetings and discussions with Artur Rocha, the project proponent, during which the current limitations of the platform and the desired improvements were

identified. From these conversations, together with the analysis of the existing architecture, a set of functional and non-functional requirements was established to guide the development of the proposed proof of concept.

3.1.1 Functional Requirements

The functional requirements define the core capabilities that the prototype must provide to demonstrate the feasibility of integrating object storage into the existing platform.

Functional Requirements		
Code	Name	Description
<i>FR01</i>	Object Storage Integration	The platform shall store newly uploaded RNA-seq datasets in MinIO instead of the local filesystem.
<i>FR02</i>	Dataset Upload	The existing upload workflow shall be adapted to transfer datasets to MinIO while maintaining the same user experience.
<i>FR03</i>	Dataset Download	Users shall be able to retrieve datasets stored in MinIO through the platform using the existing download mechanisms.
<i>FR04</i>	Metadata Management	The application shall maintain metadata associated with each stored dataset, including the information required to locate and manage the corresponding object in MinIO.
<i>FR05</i>	API Extension	The backend API shall provide the endpoints necessary to upload, download, and manage datasets stored in the object storage service.
<i>FR06</i>	Compatibility with Existing Functionality	The integration shall preserve the platform's existing dataset management workflows, requiring minimal changes to the frontend.
<i>FR07</i>	Feedback of the system state	The system must provide visual feedback (e.g., progress bars) during massive file uploads to inform the user of the ongoing status.

Table 1: Functional Requirements for development

3.1.2 Non-Functional Requirements

The non-functional requirements describe the quality attributes that the proposed solution should satisfy.

Non-Functional Requirements

Code	Name	Description
<i>NFR01</i>	Scalability	The storage solution should allow the platform to scale beyond the storage limitations imposed by the application server's local filesystem.
<i>NFR02</i>	Performance	The system should efficiently handle the upload and download of large RNA-seq datasets while minimizing the impact on the application server.
<i>NFR03</i>	Reliability	The storage architecture should improve data reliability by taking advantage of the redundancy and fault-tolerance mechanisms provided by MinIO.
<i>NFR04</i>	Maintainability	The storage layer should be modular and sufficiently decoupled from the application logic to facilitate future maintenance and extensions.
<i>NFR05</i>	Interoperability	The solution should rely on the Amazon S3 API to ensure compatibility with existing tools and facilitate potential future migration to other S3-compatible storage providers.
<i>NFR06</i>	Transparency	The migration from filesystem storage to object storage should be transparent to end users, preserving the existing workflow and user interface.

Table 2: Non-Functional Requirements for development

3.1.3 Constraints

The scope of this work is limited to the development of a proof of concept intended to evaluate the feasibility of integrating object storage into the existing platform. Consequently, the solution was designed under the following constraints:

Constraints

Code	Description
<i>C01</i>	The existing FastAPI backend and Angular frontend were to be preserved, introducing only the modifications necessary to support object storage and slight changes to data association.
<i>C02</i>	MinIO was selected as the object storage system as it is extremely cost effective.
<i>C03</i>	SQLite remained the database management system used throughout the development of the prototype.
<i>C04</i>	The implementation focused on validating the architectural approach rather than providing a production-ready deployment, with emphasis placed on demonstrating functional integration and compatibility with the existing platform.

Table 3: Constraints for development

3.2 Architecture and Technologies

An important part of the project involved studying and understanding the existing architecture of the platform, particularly its current filesystem-based storage approach and the API endpoints responsible for dataset management. This analysis was essential for identifying the components that would require modification and for designing an integration strategy that would preserve the platform’s existing functionality while introducing a more scalable storage solution.

3.2.1 Technology Stack

With this understanding, the proposed solution builds upon the existing architecture by introducing an object storage layer through the integration of MinIO. The backend is implemented using **FastAPI** [12], a modern Python web framework that provides high performance and automatically generates API documentation through **OpenAPI (Swagger)** [13]. Dependency management and the Python development environment are handled using **uv** [14], which simplifies package management and virtual environment creation while improving installation speed and ensuring reproducible development setups. The frontend is developed with **Angular** [15], while the desktop application is packaged using **Electron** [16], enabling cross-platform deployment with a native desktop feel. Although a desktop port was not defined as a core requirement, it was identified as a valuable addition. Given the technical synergy between Angular and Electron frameworks, its implementation was pursued to further enhance the platform’s versatility.

To support scalable file storage, the project integrates **MinIO** [5], an S3-compatible object storage system. Communication with the storage service is performed using the **Amazon S3 API** [6], allowing the application to manage objects through a standardized and widely adopted interface. During development, **SQLite** [17] is used as the database management system to store application data and file metadata.

The application and its supporting services are containerized using **Docker** [18], ensuring a consistent and reproducible development environment while simplifying deployment. Finally, **GitLab** [7] is used for version control, collaborative development, and project management throughout the implementation of the prototype.

3.2.2 Implementation

The implementation phase focused on translating the proposed architectural shifts into a functional system. The primary goal was to decouple the file storage from the application server while preserving the integrity of the existing frontend, backend, and database structures. This required adapting the dataset lifecycle so that files are managed through the S3-compatible service, while the application retains absolute control over metadata, user authorization, and logical dataset associations.

Backend Storage Service and Metadata Management To ensure the system’s maintainability and adhere to software engineering best practices, a dedicated storage service was introduced within the FastAPI backend. This service acts as the sole integration point with the MinIO API. By centralizing operations such as generating presigned URLs, verifying object existence, and managing multipart uploads within a specific service, the storage logic remains highly modular. This architectural decision prevents low-level storage calls from leaking into the broader backend application logic.

Simultaneously, the dataset database model was extended to store the information required to locate and manage each file within the object store. New fields were introduced to track the bucket name, specific storage path, file size, and upload status. Consequently, the SQLite database acts as the source of truth for the logical state of each dataset, while the physical file contents are handled entirely by the storage layer.

Upload and Download Coordination Workflow To avoid saturating the application server’s I/O, the upload and download workflows were designed around a direct client-to-storage architecture.

When a user initiates an upload, the frontend first requests an operation from the backend. The backend validates the request, creates an initial dataset record, checks for potential conflicts, and returns the cryptographic signatures (presigned URLs) necessary for the frontend to upload the file directly to MinIO. Once the transfer concludes, the frontend notifies the backend, which verifies the newly stored object and finalizes the dataset’s status. The download workflow follows an identical coordination model, with the backend granting secure, temporary access URLs to authenticated users.

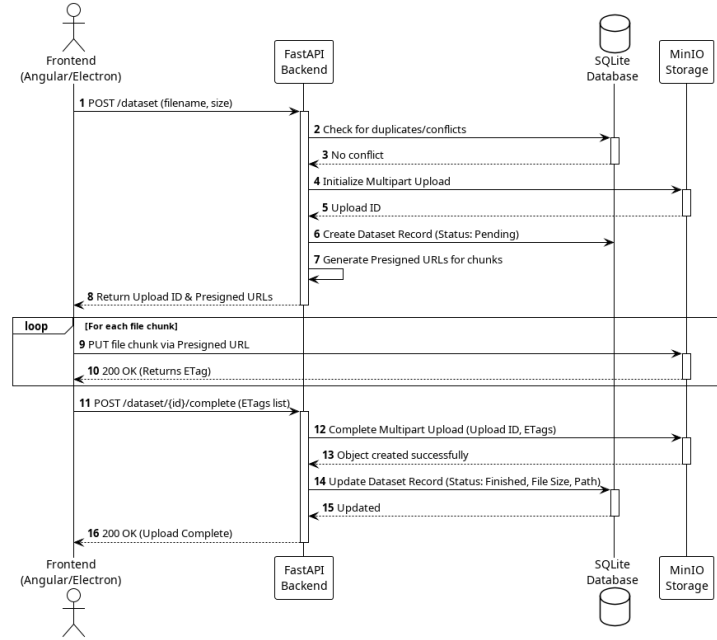


Figure 2: UML Sequence Diagram illustrating the coordinated upload workflow between the Angular frontend, FastAPI backend, and MinIO storage layer.

Technical Difficulties and Resolutions During development, several technical challenges inherent to managing large datasets were encountered. Resolving these required implementing specific fault-tolerance mechanisms:

- Large File Instability:** Transferring datasets exceeding 50 GB in a single continuous stream proved highly susceptible to network timeouts and instabilities. *Resolution:* A **Multipart Upload** architecture was implemented. The frontend divides the file into smaller chunks, uploading each independently. The storage service returns an *ETag* for each successful part. Upon completion, the frontend submits the ordered list of ETags to the backend, which commands MinIO to assemble the final object.
- Session Interruptions:** Accidental tab closures or browser refreshes during lengthy uploads resulted in lost progress and orphaned data. *Resolution:* A **Resumable Upload** mechanism was engineered. The backend persists multipart session data in the database, while the frontend caches recovery information locally. If an upload is interrupted, the system detects the pending session upon the user's return. By comparing the parts already received by the storage service, the frontend uploads only the missing chunks, saving substantial time. If the user discards the upload, the backend securely aborts the operation and clears the session.

- **Concurrency and Overwrites:** Replacing existing files without user confirmation posed a risk of accidental data loss. *Resolution:* A **Conflict Handling** flow was added. Before creating or moving an object, the backend pre-checks the target location. If occupied, a **409 Conflict** is returned, prompting the frontend to display a confirmation modal. The operation only proceeds upon explicit user authorization.

Logical Organization and Sample Association Finally, integrating the new storage paradigm required compatibility with the platform’s existing biological sample workflows. When a dataset is logically associated with a sample via the frontend, the backend updates the database relationships and physically relocates the object in MinIO from an initial */Unassigned* bucket to the appropriate sample-specific path. When disassociated, the object is moved back. This approach ensures that the physical storage organization continuously mirrors the logical architecture expected by the researchers.

Desktop Client Port (Electron) Although not strictly defined as a core project requirement, a standalone desktop application was developed to enhance the platform’s accessibility and provide a native user experience. The port was achieved by wrapping the existing Angular frontend within an Electron container.

To support this deployment without altering the web-based architecture, dedicated environment configurations were introduced specifically for the Electron build. This allowed the desktop client to correctly route API requests to the FastAPI backend while maintaining the same underlying codebase.

During the porting process, several distinct technical challenges were encountered and resolved:

- **Standalone Authentication:** A major technical hurdle involved the Keycloak authentication flow. Standard OAuth/OIDC browser redirects are often obstructed or mishandled within Electron’s isolated native window context. This was resolved by refactoring the authentication mechanism to properly capture and manually handle Keycloak redirect URIs and tokens within the desktop application. This successfully established a functional authenticated session, providing the necessary stability for the current proof of concept
- **Native UI Interactions:** Integrating web-native user interface components into a desktop window required specific adjustments. Challenges arose regarding the reliable rendering of alert popups and file overwrite confirmation modals. While the core features were successfully stabilized to function within the native shell, handling complex native dialogs remained highly sensitive, identifying an area for further refinement in future iterations.

Final High Level Architecture To sum up, there is a high level diagram of the achieved architecture.

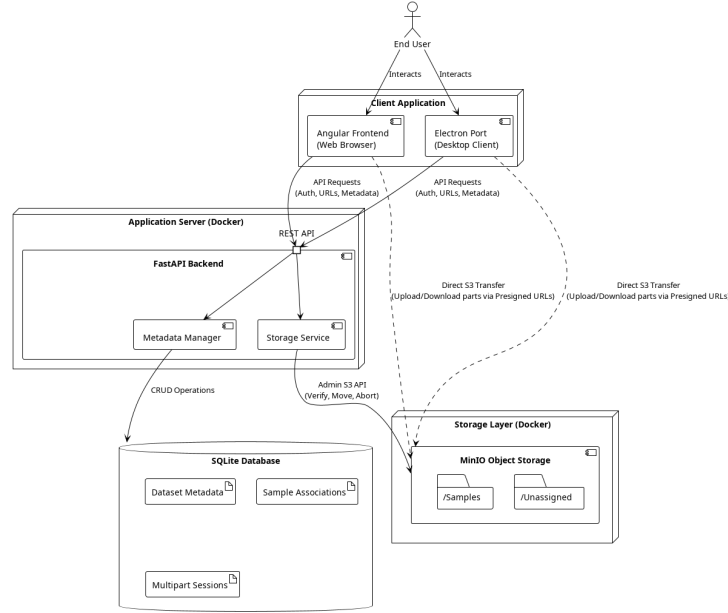


Figure 3: High-level system architecture illustrating the components of the platform and the direct client-to-storage communication path.

3.3 Developed Solution

The platform’s existing look and feel were preserved, ensuring that researchers do not need to learn a new workflow to manage their data. However, from the user’s perspective, the system now provides significantly enhanced visual feedback, safer data handling through warning modals, and robust recovery options.

The following subsections present the developed solution from the user’s point of view, illustrating the primary workflows with screenshots of the updated interface.

3.3.1 Dataset Upload and Visual Feedback

The primary interaction point is the dataset upload interface. When a user wishes to upload a new RNA-seq dataset, they navigate to the dataset management page and select their file. Once the upload begins, a dynamic progress bar is displayed, informing the user of the ongoing system state and the percentage of the file that has been successfully transferred.

3.4 Validation

The prototype was validated through manual functional testing of the main dataset management workflows. The validation process focused on verifying that the integration of MinIO object storage preserved the existing behavior of the platform while introducing support for more robust upload and storage mechanisms.

The first group of tests focused on the basic upload and download workflow. Small datasets were uploaded through the frontend and then verified in MinIO. The corresponding database records were also inspected to confirm that meta-data such as filename, storage path, file size, bucket name, and upload status were correctly stored. The download operations were then tested to confirm that the users could retrieve files stored in MinIO through the platform using the existing dataset access workflow.

A second group of tests validated conflict and overwrite handling. The same dataset was uploaded more than once to verify that the backend detected existing objects or dataset records and returned a 409 Conflict response. The frontend was then expected to display a confirmation modal, ensuring that overwrite operations only occurred after explicit user confirmation.

The sample association and disassociation workflows were also tested. Datasets were associated with samples and later disassociated, verifying that the corresponding database associations were updated correctly and that the related objects were moved to the expected storage paths in MinIO. This ensured that the new object-storage architecture remained compatible with the platform's existing sample, read, and annotation workflows.

Finally, multipart and resumable uploads were tested using larger files. These tests confirmed that large datasets were divided into multiple parts, uploaded directly to MinIO, completed successfully using the *ETags* returned by the storage service, and marked as finished in the database. The resume mechanism was tested by refreshing the browser during an upload, returning to the dashboard, selecting the same file again, and verifying that only the missing parts were uploaded before completing the multipart upload.

Test cases		
Name	Expected Result	Result
<i>Small dataset upload</i>	The dataset is uploaded to MinIO and marked as finished in the database.	Passed
<i>Dataset download</i>	The user can retrieve the file stored in MinIO through the platform.	Passed
<i>Duplicated upload</i>	The backend detects the conflict and returns a 409 Conflict response.	Passed
<i>Confirmed overwrite</i>	The existing object is overwritten only after explicit user confirmation.	Passed
<i>Dataset association with sample</i>	The object is moved to the expected sample storage path and the database association is created.	Passed
<i>Dataset disassociation from sample</i>	The object is moved back to <i>/Unassigned</i> and the database association is removed.	Passed
<i>Multipart upload</i>	The large file is split into parts, uploaded to MinIO, completed successfully, and then marked as finished.	Passed
<i>Resume after browser re-fresh</i>	The pending upload session is detected and the upload continues from the already uploaded parts.	Passed
<i>Wrong file selected during resume</i>	The upload is not aborted, the session is preserved, and the user can retry with the correct file.	Passed
<i>Discard interrupted upload</i>	The backend aborts the multipart upload, clears the session, and prevents stale processing records.	Passed

Table 4: Test cases with descriptions and results

The validation results are summarized in the previous table. Overall, the tests confirmed that the developed proof of concept satisfies the main functional requirements and demonstrates the feasibility of adopting an object-storage-based architecture for the platform. It is also important to note that, besides the said tests cases, it was also discussed during the weekly meetings with the proponent whether the functionalities were aligned with what was initially idealized.

4 Conclusions

This final chapter concludes the report by summarizing the overall outcomes of not only integrating the MinIO object storage solution into the VAXPRED platform, but also addressing additional objectives, like the multipart upload and the desktop port. It begins by evaluating the results achieved against the initial objectives of the project, providing a critical evaluation of the functional prototype. Following this, the lessons learned throughout the development process are discussed, reflecting on the technical and collaborative insights gained by the team. The chapter then identifies potential avenues for future work, outlining known limitations and areas where the system’s architecture and functionality could be further expanded. Finally, it reinforces the project’s broader social im-

pact by mapping the completed work to the relevant United Nations Sustainable Development Goals.

4.1 Achieved Results

The project resulted in a functional proof of concept that demonstrates the integration of object storage into the VAXPRED platform. The implemented prototype successfully adapts the dataset management workflow to use MinIO while preserving the main behavior expected by users and maintaining compatibility with the existing frontend, backend, and database structure.

The main outcome was the creation of a storage workflow in which dataset files are no longer handled as direct filesystem resources by the application server. Instead, the backend coordinates storage operations, manages metadata, validates user actions, and provides the frontend with the information required to upload, download, relocate, or remove files. This confirms that the platform can adopt a more modular storage architecture without requiring a full redesign of the existing application.

Beyond the basic storage integration, the prototype also introduced mechanisms that make large-file handling more reliable. Multipart upload allows large datasets to be transferred in smaller parts, while the resumable upload mechanism provides recovery from interruptions such as browser refreshes or page closures. Together with overwrite confirmation and abort handling, these features improve the robustness and predictability of the dataset workflow.

Overall, the achieved results show that the proposed approach is technically feasible and compatible with the needs of the existing platform. Although the implementation remains a proof of concept, it provides a solid foundation for future production-oriented improvements, such as distributed deployment, stronger validation mechanisms, and more extensive automated testing.

Lastly, regarding the contribution of each group member, the qualitative value of each team member’s work was uniform, with all individuals delivering work essential to the prototype’s success. In quantitative terms, the overall workload was also distributed equally, with Artur Moreira, Casemiro Medeiros, Guilherme Sá, and José Vizcaíno each contributing 25%.

4.2 Learned Lessons

One of the main lessons learned during the project was the importance of separating file storage from application logic. Keeping dataset metadata in the database while storing the actual files in MinIO makes the system easier to scale and maintain, but it also requires careful coordination between the backend, the database, the frontend, and the storage service.

The project also showed that integrating a new architectural component into an existing codebase is not only a technical task, but also a compatibility challenge. Since VAXPRED already had established dataset, sample, read, and annotation workflows, the new storage solution had to be introduced incrementally and without breaking the expected behavior of the platform.

Another important lesson was that large-file upload support introduces additional failure scenarios that must be handled explicitly. Multipart upload improves robustness, but it also requires managing upload sessions, validating resumed files, collecting ETags, completing uploads correctly, and cleaning up interrupted transfers.

Finally, the team reinforced the value of iterative development and regular feedback. Weekly meetings with the supervisors helped clarify requirements, validate technical decisions, and adjust the implementation according to the needs of the platform and the expectations of INESC TEC.

4.3 Known Limitations

Although the developed prototype demonstrates the feasibility of integrating object storage into the VAXPRED platform, the implementation still presents some limitations. First, the solution was developed and tested in a local environment, with MinIO deployed through Docker Compose. As a result, the prototype does not yet represent a production-ready storage deployment, particularly regarding distributed storage, replication, monitoring, and operational management.

Another limitation concerns the resumable upload mechanism. The current implementation supports the recovery of one pending multipart upload session at a time, using local browser storage to identify interrupted uploads. This is sufficient for validating the concept, but it does not yet cover more complex scenarios such as multiple simultaneous interrupted uploads, recovery across different browsers or devices, or cases where the user clears local browser data.

The resume validation process is also limited. At this stage, the selected file is checked using information such as filename, file size, and last modification date. While this provides a practical validation mechanism for the prototype, it does not offer the same level of certainty as a checksum or hash-based comparison, which would be more appropriate for a production environment.

In addition, the current multipart upload implementation focuses on correctness and recovery rather than maximum transfer performance. Parts are handled in a controlled manner, but further optimization could be achieved by introducing parallel part uploads with concurrency limits, better progress persistence, and more detailed error recovery strategies.

Furthermore, an additional limitation is associated with the desktop client port developed using Electron. While the core functionalities of the platform are operational within this standalone environment, the current implementation lacks the stability and reliability required for a full-scale deployment. At this stage, it serves as a functional proof of concept rather than a production-ready application, requiring further refinement, performance profiling, and robust cross-platform testing before it can be safely distributed to end-users.

Finally, validation was mainly performed through manual functional testing. Although these tests covered the main workflows implemented during the project, a production-ready solution would require a more complete automated

test suite, including integration tests for storage failures, interrupted uploads, stale sessions, overwrite conflicts, and edge cases involving large datasets.

4.4 Future Work

Although the developed prototype validates the feasibility of integrating object storage into the platform, several improvements could be made in future work, especially the known limitations addressed previously.

First, MinIO should be deployed in a distributed or replicated configuration instead of being used only as a local Docker-based development service. This would allow the platform to benefit from stronger redundancy and production-level fault tolerance.

Second, the Electron-based desktop port requires additional refinement. Although the core functionalities are operational, the current implementation lacks the stability and reliability required for a production release.

Third, the resumable upload mechanism could be extended to support multiple simultaneous uploads, since the current implementation focuses on one interrupted multipart upload session at a time.

Fourth, upload progress could be stored in a global frontend service so that the progress bar remains visible even when the user navigates away from the dataset page and later returns.

Fifth, file validation during resume could be improved using hashes or checksums instead of relying only on filename, file size, and last modification date.

Sixth, multipart upload performance could be improved by uploading several parts in parallel with a controlled concurrency limit. Finally, automated integration tests should be added to validate scenarios such as MinIO unavailability, duplicate uploads, interrupted multipart uploads, resume, abort, and stale session cleanup.

4.5 Related Sustainable Development Goals

The work developed in the VAXPRED project supports the United Nations (UN) 2030 Agenda by addressing specific Sustainable Development Goals (SDGs). First, the platform directly aligns with SDG 3, which advocates for healthy lives and well-being for all at all ages, including a specific subgoal to support the research and development of vaccines [19]. Additionally, exploring more scalable alternatives for the initial filesystem-based storage approach contributes to SDG 9, which focuses on building resilient infrastructure, promoting inclusive and sustainable industrialization and fostering innovation [19]. Finally, VAXPRED also intersects with SDG 12, which aims to ensure sustainable consumption and production patterns [19]. The platform’s ability to predict whether a vaccine candidate is effective and more likely to succeed in later stages of vaccine development [3] helps prevent the manufacturing of unviable vaccines. This ultimately results in the more sustainable management of resources and raw materials.

References

- [1] INESC TEC. Inesc tec - institute for systems and computer engineering, technology and science. <https://www.inesctec.pt/en/institution>, 2026. [Online; accessed 28 June 2026].
- [2] INESC TEC. Human-centered computing and information science (humanise). <https://www.inesctec.pt/en/centres/humanise>, 2026. [Online; accessed 28 June 2026].
- [3] Inno4Vac. Subtopic 1: Vaxpred. <https://www.inno4vac.eu/st1vaxpred>, 2026. [Online; accessed 28 June 2026].
- [4] INESC TEC. A cloud-based systems-immunology platform for reliable predictions of vaccine efficacy. <https://www.inesctec.pt/en/projects/inno4vac>, 2026. [Online; accessed 28 June 2026].
- [5] MinIO, Inc. Minio — high performance data store for ai & analytics. <https://www.min.io/>, 2026. [Online; accessed 29 June 2026].
- [6] Amazon Web Services. Amazon simple storage service (s3) rest api. <https://docs.aws.amazon.com/AmazonS3/latest/API/Welcome.html>, 2026. [Online; accessed 30 June 2026].
- [7] GitLab. Finally, ai for the entire software lifecycle. <https://about.gitlab.com/>, 2026. [Online; accessed 29 June 2026].
- [8] IBM. What is a monolithic architecture? <https://www.ibm.com/think/topics/monolithic-architecture>, 2026. [Online; accessed 29 June 2026].
- [9] Saurabh Gupta. Decoupled architecture — microservices. <https://medium.com/@saurabh.engg.it/decoupled-architecture-microservices-29f7b201bd87>, 2023. [Online; accessed 29 June 2026].
- [10] Mike Bayer. Alembic. <https://alembic.sqlalchemy.org/>, 2026. [Online; accessed 04 July 2026].
- [11] Red Hat. Keycloak. <https://www.keycloak.org/>, 2026. [Online; accessed 04 July 2026].
- [12] Sebastián Ramírez. Fastapi. <https://fastapi.tiangolo.com/>, 2026. [Online; accessed 30 June 2026].
- [13] OpenAPI Initiative. Openapi initiative. <https://www.openapis.org/>, 2026. [Online; accessed 01 July 2026].
- [14] Astral. uv: An extremely fast python package and project manager. <https://docs.astral.sh/uv/>, 2026. [Online; accessed 01 July 2026].

- [15] Google. Angular - the modern web developer's platform. <https://angular.dev/>, 2026. [Online; accessed 30 June 2026].
- [16] OpenJS Foundation. Electron — build cross-platform desktop apps with javascript, html, and css. <https://www.electronjs.org/>, 2026. [Online; accessed 02 July 2026].
- [17] SQLite Consortium. Sqlite home page. <https://www.sqlite.org/>, 2026. [Online; accessed 30 June 2026].
- [18] Docker Inc. Docker: Accelerated container application development. <https://www.docker.com/>, 2026. [Online; accessed 01 July 2026].
- [19] United Nations. Transforming our world: the 2030 agenda for sustainable development. <https://sdgs.un.org/2030agenda>, 2015. [Online; accessed 02 July 2026].